

Elixir style: source comparison

Generated with Elixir 1.19.5. A guide column shows "Yes" where it proposes the rule; the mix format column shows "Enforced" where the formatter applies it automatically. 40 master rules.
Sources a mix format = mix format; Lexmag = Lexmag's Elixir Style Guide; Credo = Credo's Elixir Style Guide; Adams = The Elixir Style Guide.
Enforced by mix format

Rule	mix format	Lexmag	Credo	Adams
Capture operator spacing	Enforced			
Collapse consecutive blank lines	Enforced			
Comments on their own line	Enforced			Yes
Fit collections on a single line when they fit	Enforced			
Group digits in large numbers	Enforced	Yes	Yes	
Lowercase exponent letter in floats	Enforced			
No parentheses around anonymous function arguments	Enforced	Yes		
No redundant parentheses	Enforced			
No semicolons between statements	Enforced	Yes	Yes	
No spaces around bitstring segment options	Enforced			
No spaces around the range operator	Enforced	Yes		Yes
No spaces inside brackets	Enforced		Yes	Yes
No trailing comma in collections	Enforced	Yes		
Omit brackets from keyword lists	Enforced			Yes
Space after commas	Enforced	Yes	Yes	Yes
Space after the comment marker	Enforced	Yes		Yes
Spaces around binary operators	Enforced	Yes	Yes	Yes
Spaces around the -> operator	Enforced			
Unix line endings	Enforced		Yes	Yes
Uppercase hexadecimal literals	Enforced	Yes		

Formatting

Rule	mix format	Lexmag	Credo	Adams
Maximum line length			Yes	Yes
No trailing whitespace		Yes	Yes	Yes
Trailing newline at end of file		Yes	Yes	Yes
Two-space indentation		Yes	Yes	

Naming

Rule	mix format	Lexmag	Credo	Adams
Avoid one-letter variable names		Yes		
CamelCase for modules		Yes	Yes	Yes
Question mark for predicate functions		Yes	Yes	Yes
snake_case for atoms, variables and functions		Yes	Yes	Yes

Modules

Rule	mix format	Lexmag	Credo	Adams
Consistent module attribute layout		Yes		Yes
Order alias/import/require alphabetically				Yes
Use the __MODULE__ pseudo-variable		Yes	Yes	Yes

Expressions

Rule	mix format	Lexmag	Credo	Adams
Avoid needless pipelines		Yes		Yes
No anonymous functions in pipelines		Yes		
No else clause with unless		Yes	Yes	Yes
Parentheses on zero-arity function calls		Yes		Yes
Use pipelines for chained calls		Yes		Yes
Use true as the last cond clause		Yes		Yes

Exceptions

Rule	mix format	Lexmag	Credo	Adams
Error suffix for exception modules		Yes	Yes	Yes
Lowercase exception messages		Yes		Yes
No trailing punctuation in exception messages		Yes		Yes

Examples

Enforced by mix format

Capture operator spacing

```
# bad
&(&1)
# good
& &1
```

Collapse consecutive blank lines

```
# bad
a = 1
b = 2
# good
a = 1
b = 2
```

Comments on their own line

```
# bad
x = 1 # set x
# good
# set x
x = 1
```

Fit collections on a single line when they fit

```
# bad
[1,
2]
# good
[1, 2]
```

Group digits in large numbers

```
# bad
1000000
# good
1_000_000
```

Lowercase exponent letter in floats

```
# bad
1.0E3
# good
1.0e3
```

No parentheses around anonymous function arguments

```
# bad
fn(x) -> x end
# good
fn x -> x end
```

No redundant parentheses

```
# bad
(1 + 2)
# good
1 + 2
```

No semicolons between statements

```
# bad
a = 1; b = 2
# good
a = 1
b = 2
```

No spaces around bitstring segment options

```
# bad
<<1 ::8>>
# good
<<1::8>>
```

No spaces around the range operator

```
# bad
1 .. 2
# good
1..2
```

No spaces inside brackets

```
# bad
[ 1, 2 ]
# good
[1, 2]
```

No trailing comma in collections

```
# bad
[1, 2, ]
# good
[1, 2]
```

Omit brackets from keyword lists

```
# bad
foo([a: 1])
# good
foo(a: 1)
```

Space after commas

```
# bad
[1,2]
# good
[1, 2]
```

Space after the comment marker

```
# bad
#comment
# good
# comment
```

Spaces around binary operators

```
# bad
1+1
# good
1 + 1
```

Spaces around the -> operator

```
# bad
fn x->x end
# good
fn x -> x end
```

Unix line endings

```
# bad
a = 1\r\nb = 2
# good
a = 1\nb = 2
```

Uppercase hexadecimal literals

```
# bad
0xabcd
# good
0xABCD
```

Formatting

Maximum line length

```
# bad
{:ok, [one, two, three, four, five, six, seven, eight, nine, ten, eleven]}
# good
{:ok,
 [one, two, three, four, five,
 six, seven, eight, nine, ten]}
```

No trailing whitespace

```
# bad
x = 1
# good
x = 1
```

Trailing newline at end of file

```
# bad
(file ends without a newline)
# good
(file ends with one \n)
```

Two-space indentation

```
# bad
if x do
:ok
end
# good
if x do
:ok
end
```

Naming

Avoid one-letter variable names

```
# bad
u = get_user()
# good
user = get_user()
```

CamelCase for modules

```
# bad
defmodule My_app do
end
# good
defmodule MyApp do
end
```

Question mark for predicate functions

```
# bad
def is_valid(x), do: true
# good
def valid?(x), do: true
```

snake_case for atoms, variables and functions

```
# bad
fooBar = 1
# good
foo_bar = 1
```

Modules

Consistent module attribute layout

```
# bad
alias App.X
use GenServer
@moduledoc "..."/>
# good
@moduledoc "..."/>
use GenServer
alias App.X
```

Order alias/import/require alphabetically

```
# bad
alias App.Zebra
alias App.Apple
# good
alias App.Apple
alias App.Zebra
```

Use the __MODULE__ pseudo-variable

```
# bad
%MyApp.User{name: n}
# good
%__MODULE__ {name: n}
```

Expressions

Avoid needless pipelines

```
# bad
x |> f()
# good
f(x)
```

No anonymous functions in pipelines

```
# bad
x |> (fn v -> v + 1 end).()
# good
x |> increment()
```

No else clause with unless

```
# bad
unless x do
  a
else
  b
end
# good
if x do
  b
else
  a
end
```

Parentheses on zero-arity function calls

```
# bad
DateTime.utc_now
# good
DateTime.utc_now()
```

Use pipelines for chained calls

```
# bad
f(g(h(x)))
# good
x |> h() |> g() |> f()
```

Use true as the last cond clause

```
# bad
cond do
x -> 1
:else -> 2
end
# good
cond do
x -> 1
true -> 2
end
```

Exceptions

Error suffix for exception modules

```
# bad
defmodule App.BadInput do
end
# good
defmodule App.BadInputError do
end
```

Lowercase exception messages

```
# bad
raise "Invalid input"
# good
raise "invalid input"
```

No trailing punctuation in exception messages

```
# bad
raise "invalid input."
# good
raise "invalid input"
```